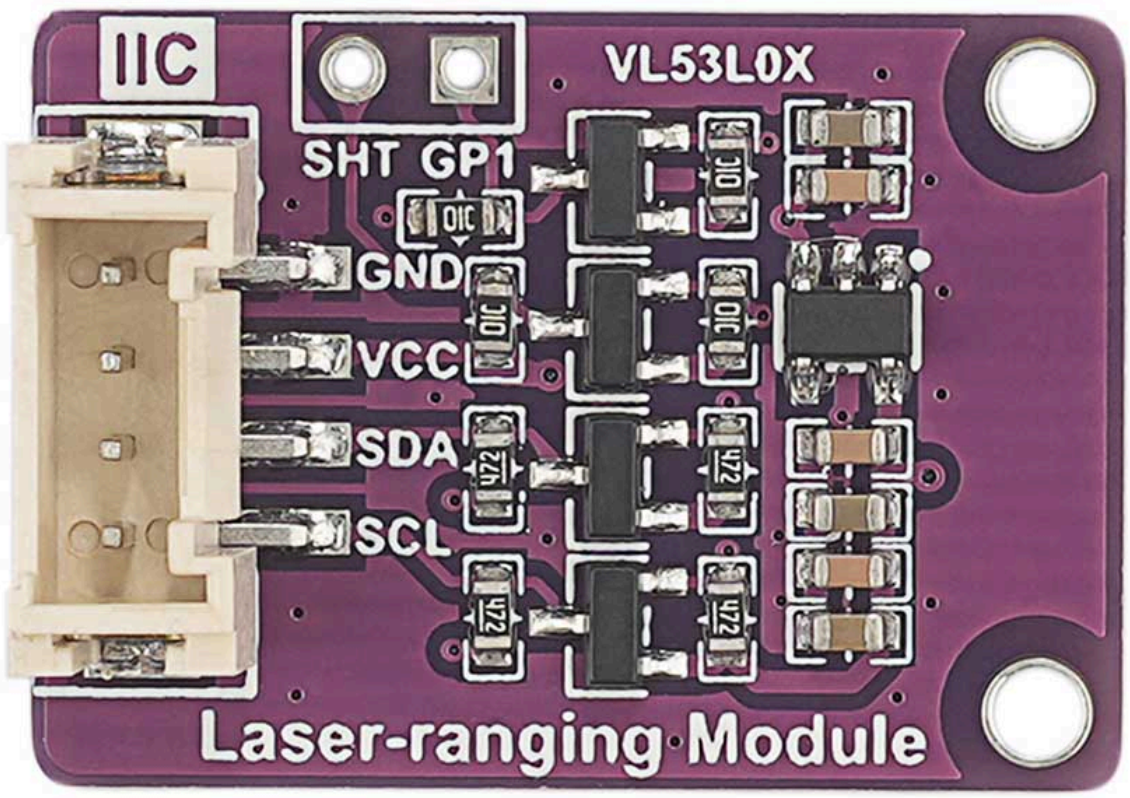
Time of Flight Laser Ranging Sensor SG S53 L0X

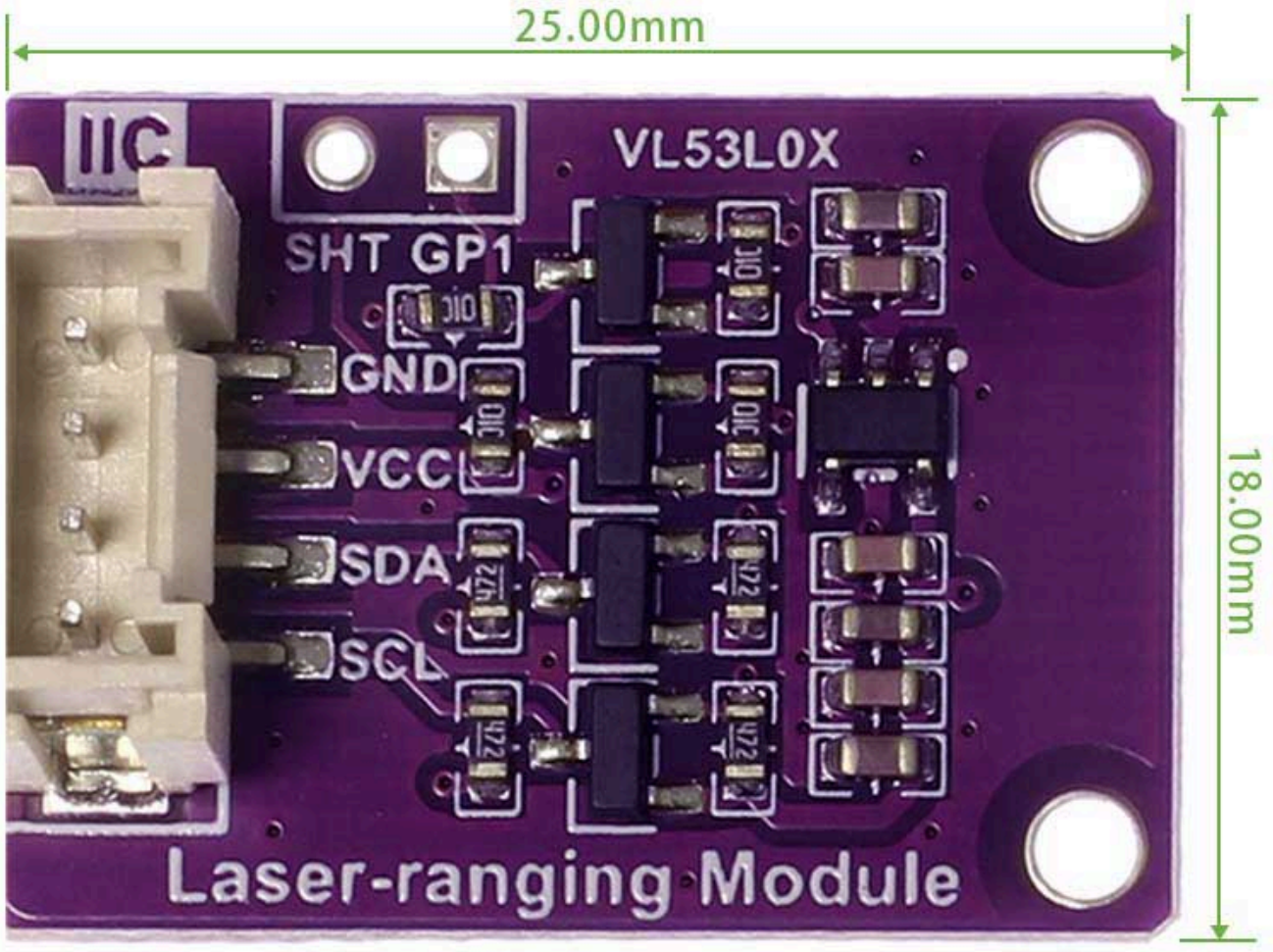


Overview

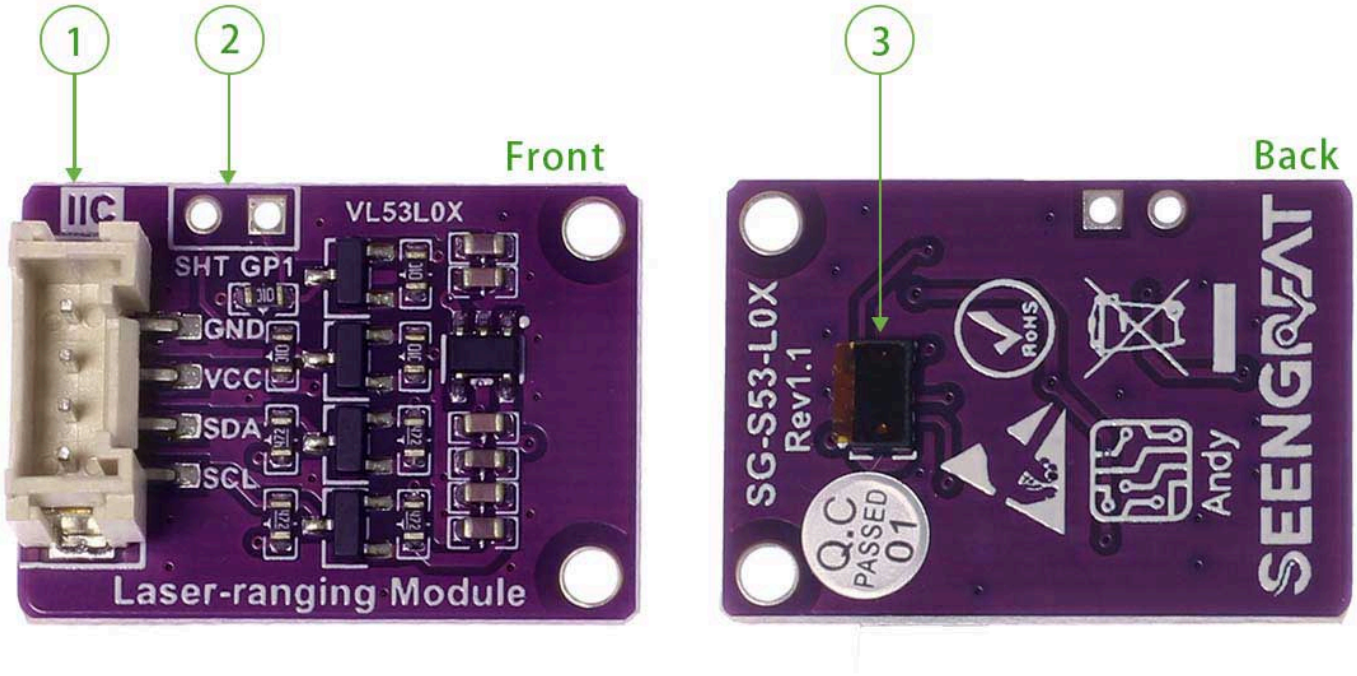
This product uses I2C interface to achieve distance measurement, with a measurement distance of up to 2 meters; Supports Raspberry Pi models, as well as Arduino and STM32; We provide C and Python versions of Raspberry Pi demo codes, as well as Arduino and STM32 versions of demo codes. The demo codes can achieve three ranging working modes: single ranging, continuous ranging, and timed ranging, as well as four ranging configuration modes: default mode, high speed, high accuracy, and long range.

Parameters

Supply voltage	3.3V/5V
Ranging distance	30~2000mm
Ranging accuracy	±5%(high speed mode)，±3%(high accuracy mode)
Ranging time (min)	20ms(short distance mode)，200ms (medium/long distance mode)
Field of view	25°
Laser wavelength	940nm
Control Chip	VL53L0X
Signal interface	I2C
Dimensions	25mm(Length) x 18mm(width)
Weight	2g



Resource Profile



① PH2.0 terminal leads out control pin

② Lead out GPIO1 and XSHUT pins

③ VL53L0X ranging chipr

Usage

Raspberry Pi demo codes usage

Hardware interface configuration description

Raspberry Pi 40Pin Expansion pins definition table

WiringPi pin	BCM GPIO	Name	Header		Name	BCM GPIO	WiringPi pin
		3.3V	1	2	5V		
8	2	SDA.1	3	4	5V		
9	3	SCL.1	5	6	GND		
7	4	GPIO.7	7	8	TXD	14	15
		GND	9	10	RXD	15	16
0	17	GPIO.0	11	12	GPIO.1	18	1
2	27	GPIO.2	13	14	GND		
3	22	GPIO.3	15	16	GPIO.4	23	4
		3.3V	17	18	GPIO.5	24	5
12	10	MOSI	19	20	GND		
13	9	MISO	21	22	GPIO.6	25	6
14	11	SCLK	23	24	CE0	8	10
		GND	25	26	CE1	7	11
30	0	SDA.0	27	28	SCL.0	1	31
21	5	GPIO.21	29	30	GND		
22	6	GPIO.22	31	32	GPIO.26	12	26
23	13	GPIO.23	33	34	GND		
24	19	GPIO.24	35	36	GPIO.27	16	27
25	26	GPIO.25	37	38	GPIO.28	20	28
		GND	39	40	GPIO.29	21	29

The module with Raspberry Pi motherboard wiring is defined in the following table:

PIN	Describe	Raspberry Pi
VCC	Power supply positive(3.3V/5V)	3.3V
GND	Power supply ground	GND
SDA	I2C data line	SDA1
SCL	I2C clock line	SCL1
SHT	shutdown control, can connects to IO pin	NC
GP1	Interrupt output, can connects to IO pin	NC

Wiringpi library installation

```
sudo apt-get install wiringpi
wget https://project-downloads.drogon.net/wiringpi-latest.deb
#Version 4B upgrade of Raspberry Pi
sudo dpkg -i wiringpi-latest.deb
gpio -v
```

If version 2.52 appears, the installation is successful

For the Bullseye branch system, use the following command:

```
git clone https://github.com/WiringPi/WiringPi
cd WiringPi
./build
gpio -v
```

Running "gpio - v" will result in version 2.70. If it does not appear, it indicates an installation error

If the error prompt "ImportError: No module named 'wiringpi'" appears when running the python version of the demo codes , run the following command

For Python 2. x version

```
pip install wiringpi
```

For Python 3. x version

```
pip3 install wiringpi
```

Note: If the installation fails, you can try the following compilation and installation:

```
git clone --recursive https://github.com/WiringPi/WiringPi-Python.git
```

Note: The -- recursive option can automatically pull the submodule, otherwise you need to

download it manually.

Enter the WiringPi Python folder you just downloaded, enter the following command, compile and install:

For Python 2. x version

```
sudo python setup.py install
```

For Python 3. x version

```
sudo python3 setup.py install
```

If the following error occurs:

```
Error:  Building this module requires either that swig is installed
        (e.g., 'sudo apt install swig') or that wiringpi_wrap.c from the
        source distribution (on pypi) is available.
```

At this time, enter the command sudo apt install swig to install swig. After that, compile and install sudo python3 setup.py install. If a message similar to the following appears, the installation is successful.

```
ges
Adding wiringpi 2.60.0 to easy-install.pth file

Installed /usr/local/lib/python3.7/dist-packages/wiringpi-2.60.0-py3.7-linux-armv7
Processing dependencies for wiringpi==2.60.0
Finished processing dependencies for wiringpi==2.60.0
```

Turn on the I2C interface

```
sudo raspi-config
```

Enable I2C interface:

```
Interfacing Options -> I2C -> Yes
```

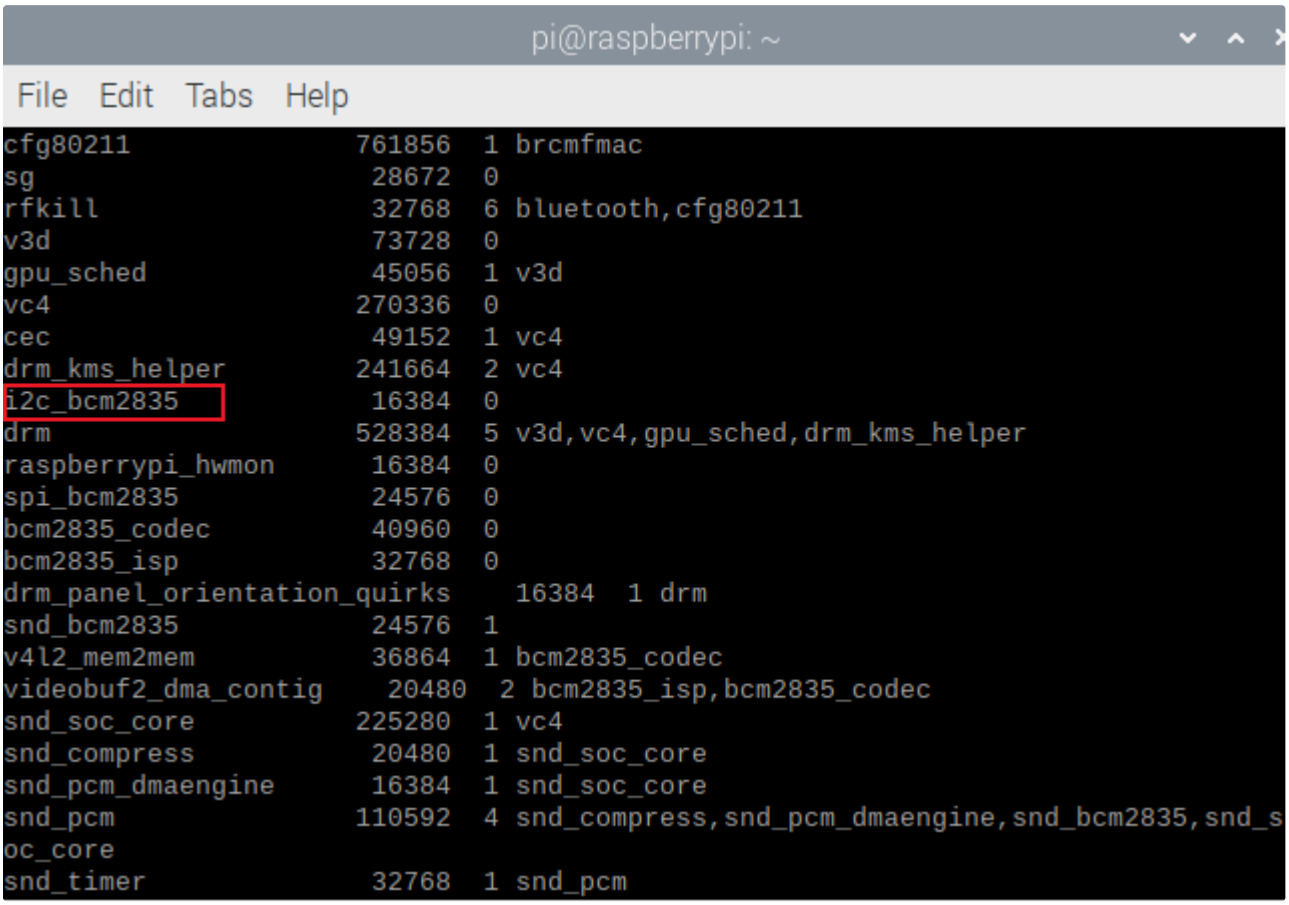
Restart the device.

```
sudo reboot
```

Run the command to check whether I2C and SPI are started.

```
lsmod
```

If i2c_bcm2835 and spi_bcm2835 are displayed, it means I2C, the SPI module is started.



Install the i2c-tools tool to confirm

```
sudo apt-get install i2c-tools
```

View connected I2C devices

```
i2cdetect -y 1
```

Check the address and indicate that the module has successfully connected to Raspberry Pi, and the displayed address is 0X29;

```
pi@raspberrypi:~ $ i2cdetect -y 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- 29 -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
pi@raspberrypi:~ $
```

Turn on the I2C interface

The demo codes uses the python 3 environment. To run the python demo codes , you need to install smbus:

```
sudo apt-get install -y python-smbus
```

2.1.5 Demo Codes

After downloading the demo codes, enter the Raspberry Pi/c directory and compile the code

```
sudo make clean
sudo make
sudo make example
```

The compiled code results are as follows.

```
pi@raspberrypi:~/SG-S53-L0X/demo_codes/Raspberry_Pi/c $ sudo make example
mkdir -p bin/
gcc -Lbin example/VL53L0X_single_example.c -lVL53L0X_c  -I. -I/home/pi/SG-S53-L0
X/demo_codes/Raspberry_Pi/c -I/home/pi/SG-S53-L0X/demo_codes/Raspberry_Pi/c/Api/
include  -o bin/VL53L0X_single_example
mkdir -p bin/
gcc -Lbin example/VL53L0X_long_distance.c -lVL53L0X_c   -I. -I/home/pi/SG-S53-L0
X/demo_codes/Raspberry_Pi/c -I/home/pi/SG-S53-L0X/demo_codes/Raspberry_Pi/c/Api/
include  -o bin/VL53L0X_long_distance
mkdir -p bin/
gcc -Lbin example/VL53L0X_continuous_example.c -lVL53L0X_c      -I. -I/home/pi/S
G-S53-L0X/demo_codes/Raspberry_Pi/c -I/home/pi/SG-S53-L0X/demo_codes/Raspberry_P
i/c/Api/include  -o bin/VL53L0X_continuous_example
mkdir -p bin/
gcc -Lbin example/VL53L0X_high_speed.c -lVL53L0X_c      -I. -I/home/pi/SG-S53-L0
X/demo_codes/Raspberry_Pi/c -I/home/pi/SG-S53-L0X/demo_codes/Raspberry_Pi/c/Api/
include  -o bin/VL53L0X_high_speed
mkdir -p bin/
gcc -Lbin example/VL53L0X_high_precision.c -lVL53L0X_c  -I. -I/home/pi/SG-S53-L0
X/demo_codes/Raspberry_Pi/c -I/home/pi/SG-S53-L0X/demo_codes/Raspberry_Pi/c/Api/
include  -o bin/VL53L0X_high_precision
```

After waiting for the program to compile, enter the bin directory

```
pi@raspberrypi:~/c/SG53LXX $ cd bin
pi@raspberrypi:~/c/SG53LXX/bin $ ls
libVL53L0X_c.a              VL53L0X_high_precision  VL53L0X_long_distance
VL53L0X_continuous_example  VL53L0X_high_speed      VL53L0X_single_example
pi@raspberrypi:~/c/SG53LXX/bin $
```

Run demo codes

```
sudo ./VL53L0X_single_example
```

Connect the module according to the demo codes description. After running the demo codes , the Raspberry Pi terminal outputs device information and measured distance (using a single ranging as an example)(Unit:mm).

```
pi@raspberrypi:~/SG-S53-L0X/demo_codes/Raspberry_Pi/c/bin $ sudo ./VL53L0X_singl
e_example

---------- SG-S53-L0X Module ----------
Raspberry Pi 4 Model B V1.0 Build 2023/4/14 10:35
VL53L0X init
VL53L0X_GetDeviceInfo:
Device name is VL53L0X ES1 or later
Device id is VL53L0CXV0DH/1$C
Device type is 238
ProductRevisionMajor is 1
ProductRevisionMinor is 1
Single measurenment distance is: 16
Single measurenment distance is: 17
Single measurenment distance is: 17
Single measurenment distance is: 16
Single measurenment distance is: 16
Single measurenment distance is: 17
Single measurenment distance is: 15
Single measurenment distance is: 19
Single measurenment distance is: 17
Single measurenment distance is: 16
Single measurenment distance is: 16
Single measurenment distance is: 17
```

Python Demo Codes

Enter the Raspberry Pi/Python directory

```
sudo make
```

Wait for the program compilation to complete, enter the example directory, and run the demo codes:

```
sudo python VL53L0X_singleexample.py
```

The routine phenomenon in the Python version is similar to that in the C language version, and the running results are shown in the following figure(Unit:mm):

```
pi@raspberrypi:~/SG-S53-L0X/demo_codes/Raspberry_Pi/python $ cd example
pi@raspberrypi:~/SG-S53-L0X/demo_codes/Raspberry_Pi/python/example $ python VL53
L0X_singleexample.py

---------- SG-S53-L0X Module ----------
Raspberry Pi 4 Model B V1.0 Build 2023/4/14 10:35
VL53L0X init
VL53L0X_GetDeviceInfo:
Device name is VL53L0X ES1 or later
Device id is VL53L0CXV0DH/1$C
Device type is 238
ProductRevisionMajor is 1
ProductRevisionMinor is 1
Single measurenment distance is: 15
Single measurenment distance is: 18
Single measurenment distance is: 16
Single measurenment distance is: 17
Single measurenment distance is: 19
Single measurenment distance is: 19
Single measurenment distance is: 18
Single measurenment distance is: 17
Single measurenment distance is: 20
Single measurenment distance is: 18
Single measurenment distance is: 18
```

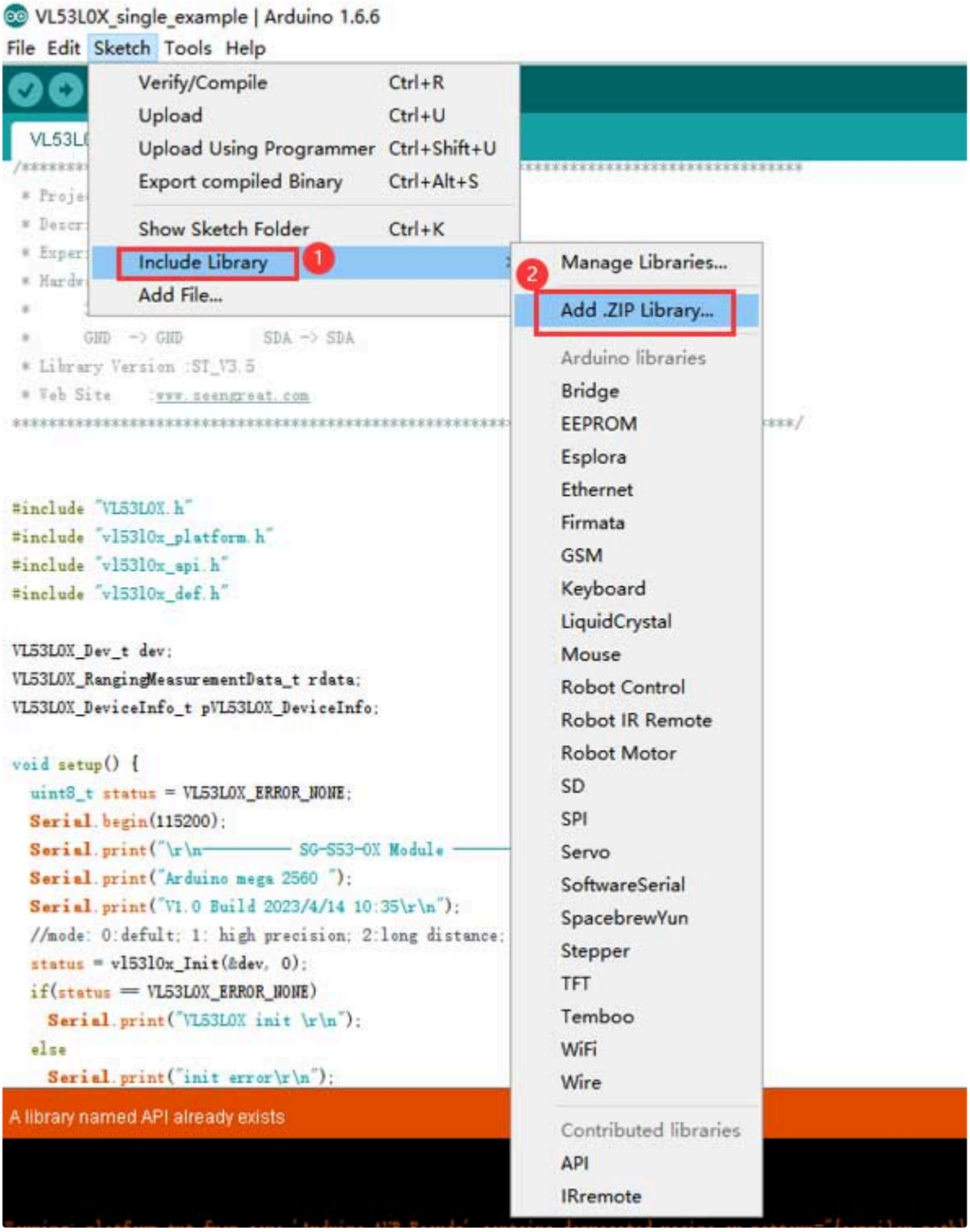
Arduino Demo Codes Usage

Wiring instructions

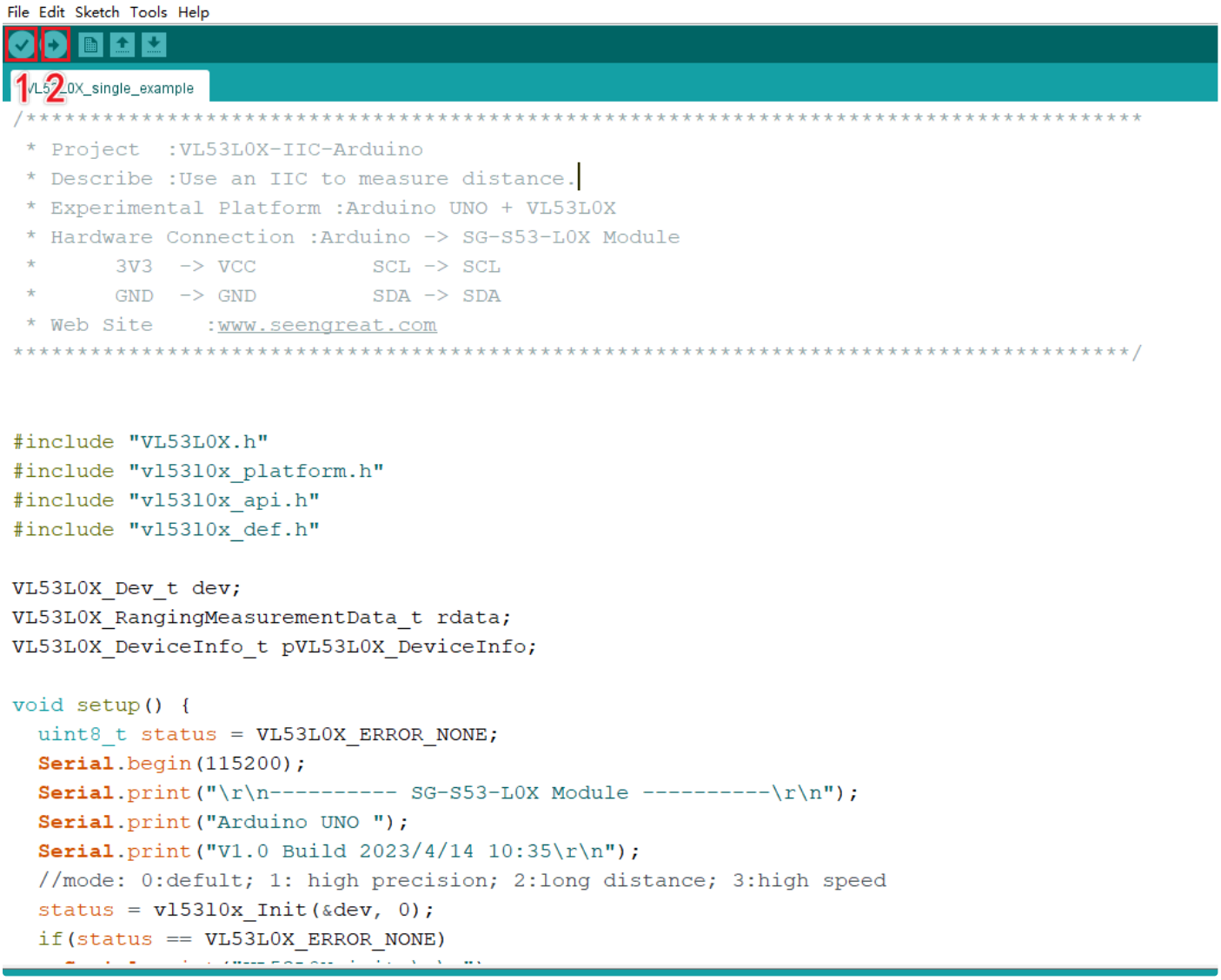
PIN	Describe	Arduino
VCC	Power supply positive(3.3V/5V)	5V
GND	Power supply ground	GND
SDA	I2C data line	SDA
SCL	I2C clock line	SCL
SHT	shutdown control, can connects to IO pin	NC
GP1	Interrupt output, can connects to IO pin	NC

Demo codes usage

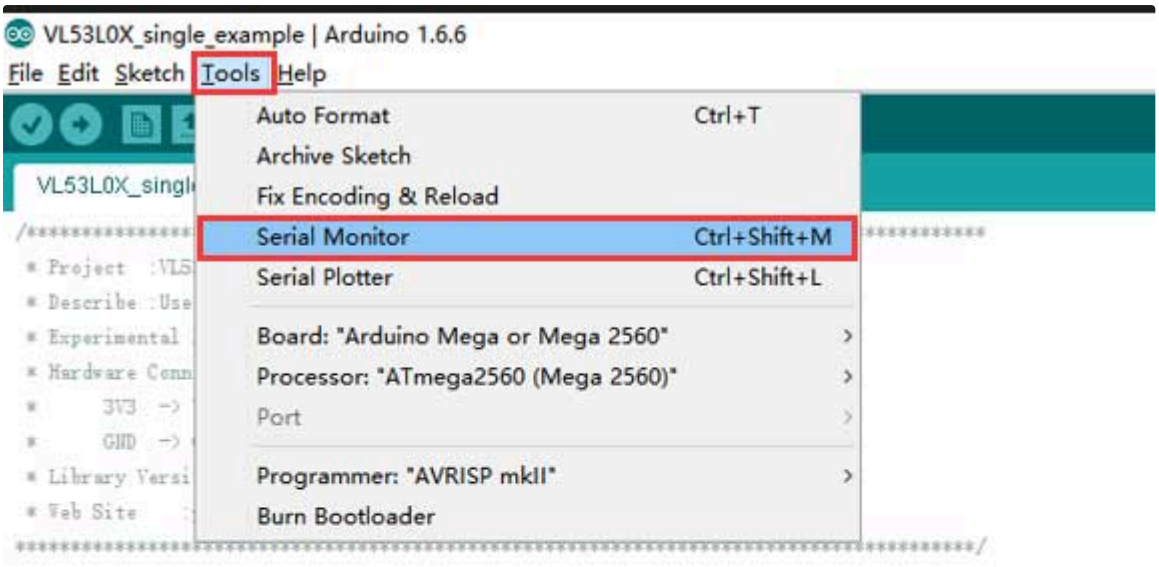
Open the project file demo_codes\Arduino\VL53L0X_single_example\VL53L0X_single_example.ino with the Arduino IDE, import API.zip from the Arduino directory into Arduino, as shown in the following figure.



Click Verify, and upload it to the development board after verification.



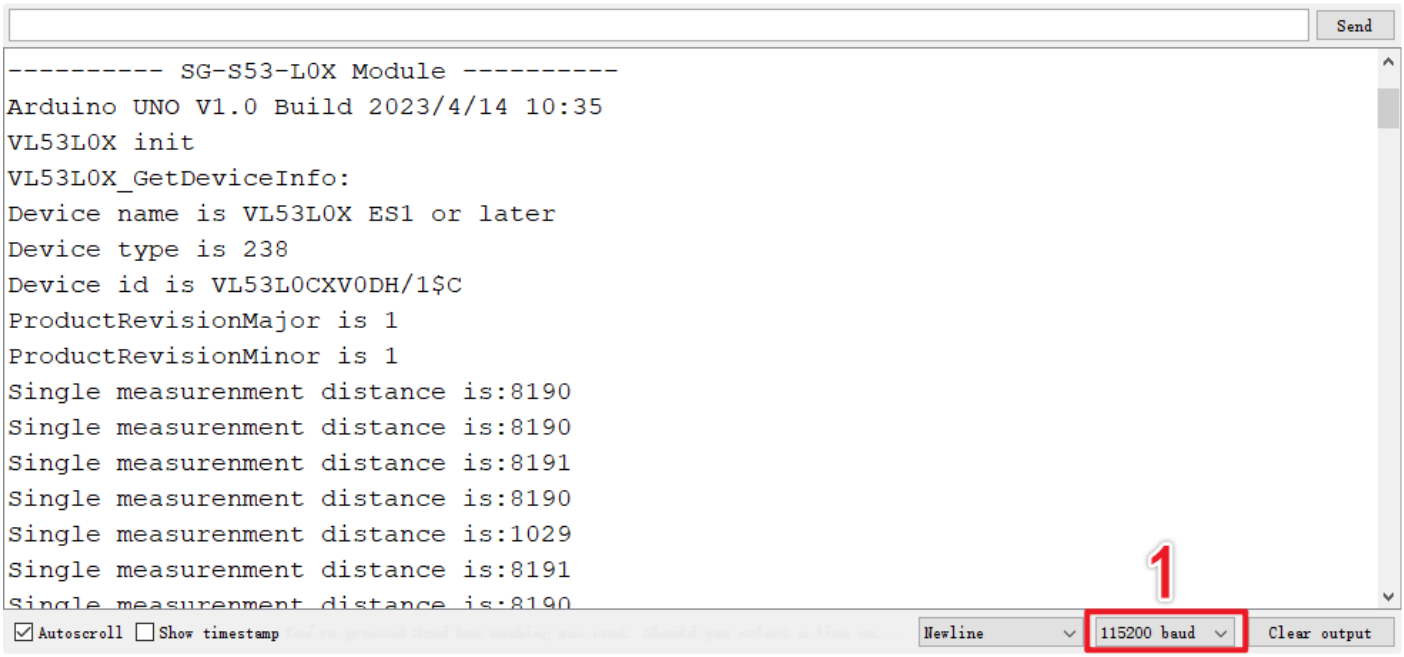
Click the tool, open the serial port monitor, set the baud rate to 115200, and observe the data changes.




```
#include "VL53L0X.h"
#include "vl53l0x_platform.h"
#include "vl53l0x_api.h"
#include "vl53l0x_def.h"

VL53L0X_Dev_t dev;
VL53L0X_RangingMeasurementData_t rdata;
VL53L0X_DeviceInfo_t pVL53L0X_DeviceInfo;
```

Open the serial port monitor and the display is as follows. Set the baud rate to 115200 (as shown in "1" in the figure below). The program running results are shown in the figure below(Unit:mm):



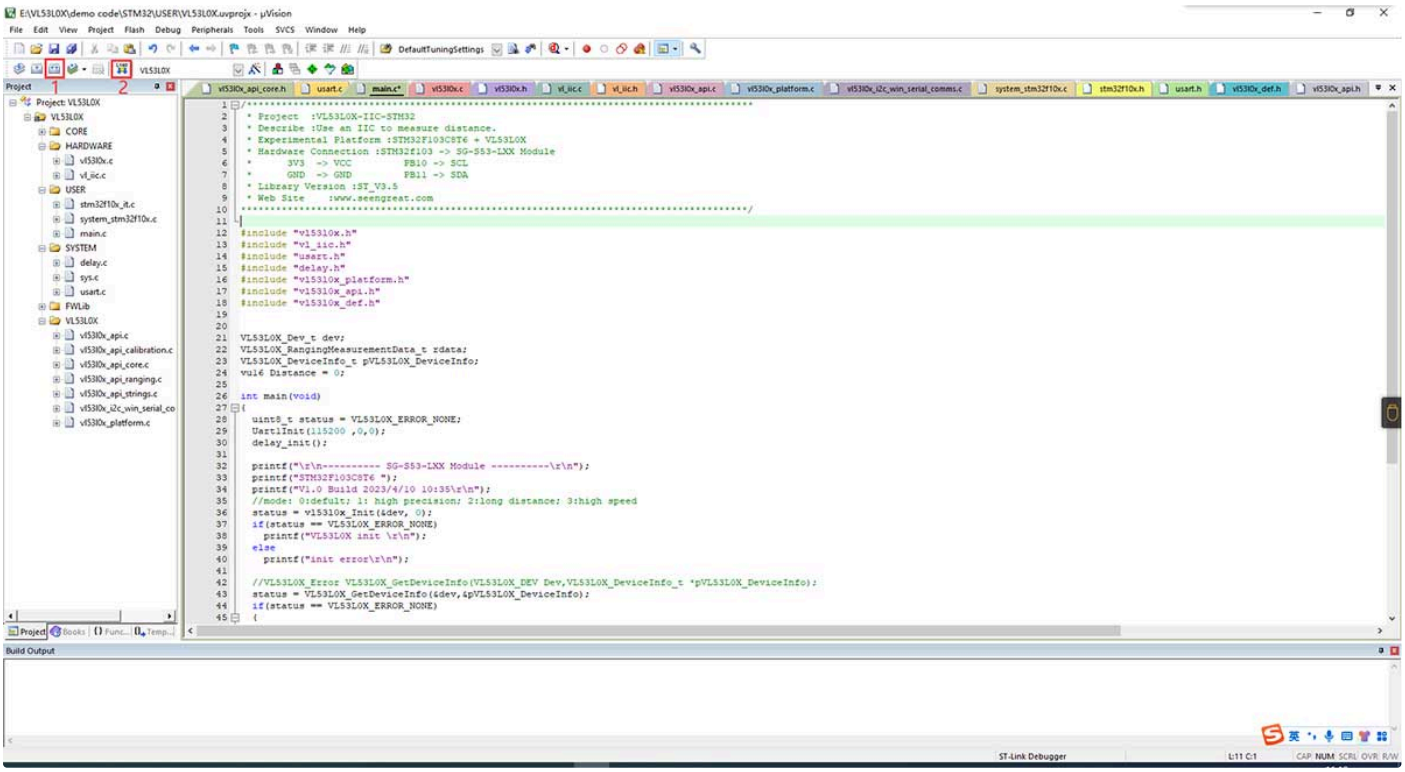
STM32 Demo Codes Usage

Wiring instructions

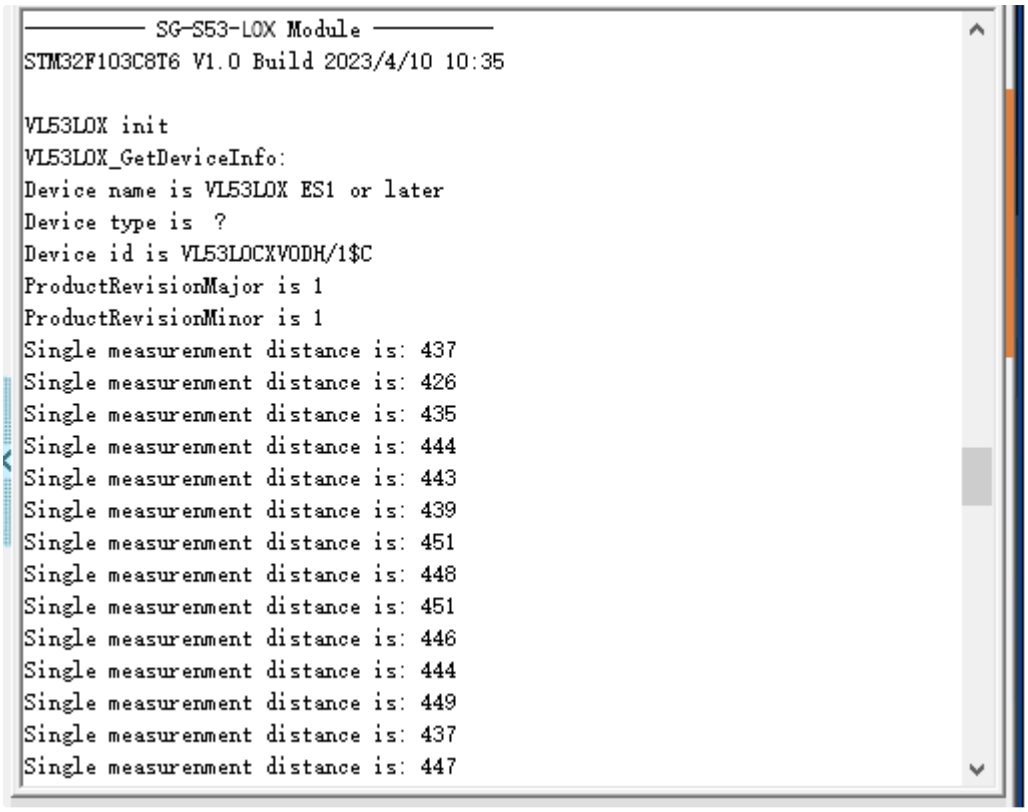
PIN	Describe	Arduino
VCC	Power supply positive(3.3V/5V)	3.3V
GND	Power supply ground	GND
SDA	I2C data line	PB11
SCL	I2C clock line	PB10
SHT	shutdown control, can connects to IO pin	NC
GP1	Interrupt output, can connects to IO pin	NC

Demo codes usage

Use Keil uVision5 software to open the VL53L0X.uvprojx project file in the directory demo_codes\STM32\USER, and connect the module with STM32 according to the above table; download the program to the STM32 development board after compiling without errors;



The debugging information output serial port in the demo codes is USART1, where PA9 is Tx and PA10 is Rx; After connecting the cable, open the Serial Port Debugging Assistant, select the serial port number, adjust the baud rate to 115200, click "Open", and the Serial Port Debugging Assistant window is shown as follows(Unit:mm):



Resources

Schematic

Product

Data Sheet

VL53L0X

Demo Codes

Technical Support

Technical Support and Product Notes